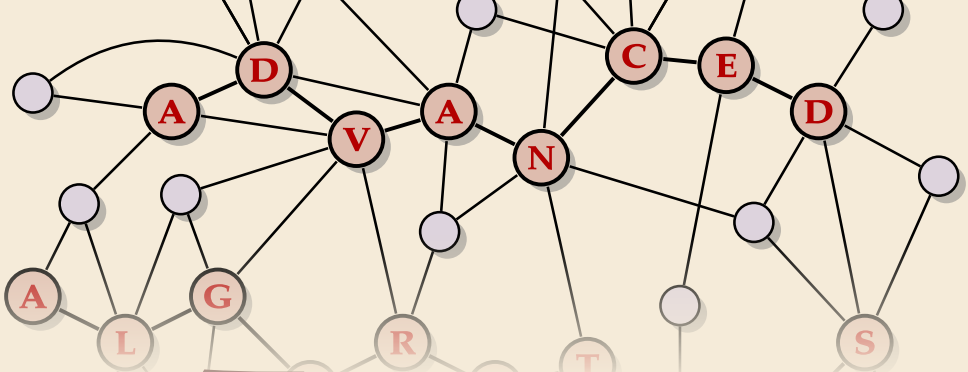
$f(k) \cdot n^{O(1)}$

# Problems w/ FPT Algorithms

- p- Vertex Cover
- p- MaxSAT
- p- Planar Independent Set
- p- Closest String

- p- variables-SAT




# 5 Parameterized Hardness

*Advanced Algorithms · Winter 2026–27*

Prof. Dr. Sebastian Wild

CS627 (Winter 2026–27)
Philipps-Universität Marburg
*version* **2026-11-09 07:50**

# Outline

## 5 Parameterized Hardness


5.1 Parameterized Reductions

5.2 Nondeterministic FPT: Para-NP

5.3 Bounded Nondeterminism: W[P]

5.4 W[1]-Hardness

5.5 W[1]-Hard Problems

# How to prove $\notin$ FPT?

- For some problems, no algorithm seems to achieve fpt running time

# How to prove $\notin$ FPT?

- For some problems, no algorithm seems to achieve fpt running time
- example: $p$-Clique

$\rightsquigarrow$ maybe no fpt algorithm can exist for $p$-Clique!

$O(n^k)$ brute force

# How to prove $\notin$ FPT?

- For some problems, no algorithm seems to achieve fpt running time
- example: $p$-Clique

$\rightsquigarrow$ maybe no fpt algorithm can exist for $p$-Clique!

- **Problem:** Certainly exists in case P = NP

$\rightsquigarrow$ strongest lower bound we can hope for will have to be conditional on P $\neq$ NP

# How to prove $\notin$ FPT?

- For some problems, no algorithm seems to achieve fpt running time
- example: $p$-Clique

$\rightsquigarrow$ maybe no fpt algorithm can exist for $p$-Clique!

- **Problem:** Certainly exists in case $\mathsf{P} = \mathsf{NP}$

$\rightsquigarrow$ strongest lower bound we can hope for will have to be conditional on $\mathsf{P} \neq \mathsf{NP}$

- Typical complexity-theory results:
  *No algorithm has property X unless (more of less widely believed) complexity hypothesis Y fails.*

# 5.1 Parameterized Reductions

# FPT Reductions

**Goal:** Compare relative hardness of *parameterized* problems

⇝ Need a notion of reductions on parameterized problems

- to preserve (non)existence of fpt algorithms, need to keep small $k$

# FPT Reductions

**Goal:** Compare relative hardness of *parameterized* problems

⇝ Need a notion of reductions on parameterized problems

- to preserve (non)existence of fpt algorithms, need to keep small $k$

## Definition 5.1 (Parameterized Reduction)

Let $(\Pi_1, \kappa_1)$ and $(\Pi_2, \kappa_2)$ be two parameterized problems, $\Pi_i = (L_i, Y_i, \Sigma_i)$.
An *fpt-reduction* (*fpt many-one reduction*) from $(\Pi_1, \kappa_1)$ to $(\Pi_2, \kappa_2)$ is a mapping $A : L_1 \to L_2$ so that for all $x \in L_1$

$n_1 = |x|$ , $k_1 = \kappa_1(x)$

1. (*equivalence*) $x \in Y_1 \iff A(x) \in Y_2$,

A running time $O(f(k_1) \cdot n_1^{\alpha(1)})$

2. (*fpt*) $A$ is computable by an fpt-algorithm (w.r.t. $\kappa_1$), and
3. (*parameter-preserving*) $\kappa_2\big(A(x)\big) \leq g\big(\kappa_1(x)\big)$ for a computable function $g : \mathbb{N} \to \mathbb{N}$.

We then write $(\Pi_1, \kappa_1) \leq_{fpt} (\Pi_2, \kappa_2)$. ◀

# Not all reductions are fpt

Many reductions from classical complexity theory are **not** parameter preserving.

**Recall:**

VertexCover
Given: graph $G = (V, E)$ and $k \in \mathbb{N}$
Question: $\exists V' \subset V \ : \ |V'| \leq k \ \wedge \ \forall \{u, v\} \in E : \big(u \in V' \vee v \in V'\big)$

IndependentSet
Given: graph $G = (V, E)$ and $k \in \mathbb{N}$
Question: $\exists V' \subset V \ : \ |V'| \geq k \ \wedge \ \forall u, v \in V' : \{u, v\} \notin E$

# Not all reductions are fpt

Many reductions from classical complexity theory are **not** parameter preserving.

**Recall:**

VertexCover
Given: graph $G = (V, E)$ and $k \in \mathbb{N}$
Question: $\exists V' \subset V \ : \ |V'| \leq k \ \wedge \ \forall \{u, v\} \in E : \big(u \in V' \vee v \in V'\big)$

IndependentSet
Given: graph $G = (V, E)$ and $k \in \mathbb{N}$
Question: $\exists V' \subset V \ : \ |V'| \geq k \ \wedge \ \forall u, v \in V' : \{u, v\} \notin E$

- We know: IndependentSet $\leq_p$ VertexCover:
  - Set $G' = G$ and $k' = |V(G)| - k$ (Complement of an indep. set must be a vertex cover, and vice versa!)

# Not all reductions are fpt

Many reductions from classical complexity theory are **not** parameter preserving.

**Recall:**

VertexCover
Given: graph $G = (V, E)$ and $k \in \mathbb{N}$
Question: $\exists V' \subset V \ : \ |V'| \leq k \ \wedge \ \forall \{u, v\} \in E : \big(u \in V' \vee v \in V'\big)$

IndependentSet
Given: graph $G = (V, E)$ and $k \in \mathbb{N}$
Question: $\exists V' \subset V \ : \ |V'| \geq k \ \wedge \ \forall u, v \in V' : \{u, v\} \notin E$

- We know: IndependentSet $\leq_p$ VertexCover:
  - Set $G' = G$ and $k' = |V(G)| - k$ (Complement of an indep. set must be a vertex cover, and vice versa!)
- $\not\Rightarrow$ $p$-IndependentSet $\leq_{fpt}$ $p$-VertexCover
  - Indeed, we know VertexCover $\in$ FPT, but IndependentSet probably isn't.

# Not all reductions are fpt

Many reductions from classical complexity theory are **not** parameter preserving.

**Recall:**

VertexCover
Given: graph $G = (V, E)$ and $k \in \mathbb{N}$
Question: $\exists V' \subset V \ : \ |V'| \leq k \ \wedge \ \forall \{u, v\} \in E : (u \in V' \vee v \in V')$

IndependentSet
Given: graph $G = (V, E)$ and $k \in \mathbb{N}$
Question: $\exists V' \subset V \ : \ |V'| \geq k \ \wedge \ \forall u, v \in V' : \{u, v\} \notin E$

- We know: IndependentSet $\leq_p$ VertexCover:
  - Set $G' = G$ and $k' = |V(G)| - k$ (Complement of an indep. set must be a vertex cover, and vice versa!)
- $\not\Rightarrow$ $p$-IndependentSet $\leq_{fpt}$ $p$-VertexCover
  - Indeed, we know VertexCover $\in$ FPT, but IndependentSet probably isn't.
- But: $p$-IndependentSet $\leq_{fpt}$ $p$-Clique (and $p$-Clique $\leq_{fpt}$ $p$-IndependentSet)
  - Set $G' = (V, \binom{V}{2} \setminus E)$ and $k' = k$ (Independent set iff clique in complement graph)

## 5.2 Nondeterministic FPT: Para-NP

# Parameterized NP: Non-deterministic NP

Good, so we have reductions.

If P corresponds to FPT . . . but what is the analogue for NP?

# Parameterized NP: Non-deterministic NP

Good, so we have reductions.

If P corresponds to FPT . . . but what is the analogue for NP?

## Definition 5.2 (para-NP)

The class para-NP consists of all parameterized decision problems that are solved by a *non-deterministic* fpt-algorithm. ◀

# Parameterized NP: Non-deterministic NP

Good, so we have reductions.

If P corresponds to FPT . . . but what is the analogue for NP?

## Definition 5.2 (para-NP)

The class para-NP consists of all parameterized decision problems that are solved by a *non-deterministic* fpt-algorithm. ◀

**Some nice properties:**

1. para-NP is closed under fpt-reductions.
2. FPT = para-NP $\iff$ P = NP
3. an analogue for *kernalization* in FPT holds for para-NP

non-trivial

# Parameterized NP: Non-deterministic NP

Good, so we have reductions.

If P corresponds to FPT . . . but what is the analogue for NP?

## Definition 5.2 (para-NP)

The class para-NP consists of all parameterized decision problems that are solved by a *non-deterministic* fpt-algorithm. ◀

**Some nice properties:**

1. para-NP is closed under fpt-reductions.
2. $\mathsf{FPT} = \text{para-NP} \iff \mathsf{P} = \mathsf{NP}$
3. an analogue for *kernalization* in FPT holds for para-NP

Can define para-NP-hard and para-NP-complete similarly as for NP:

## Definition 5.3 (para-NP-hard)

$(\Pi, \kappa)$ is para-NP-hard if $(\Pi', \kappa') \leq_{fpt} (\Pi, \kappa)$ for all $(\Pi', \kappa') \in$ para-NP. ◀

# Hello hardness, my old friend

**Theorem 5.4 (para-NP-complete → NP-hard for bounded parameter)**

Let $(\Pi, \kappa)$ be a nontrivial ($\emptyset \neq Y \neq L$) parameterized problem that is para-NP-complete. Then there is a $d \in \mathbb{N}$ such that $\Pi_{\kappa \leq d}$ ($\Pi$ restricted to instances with $\kappa(x) \leq d$) is NP-hard. ◀

The converse is essentially also true (using a generalization of kernelizations).

→ rules out that p-Clique para-NP-complete!

because $n^k$ is poly. for $k = O(1)$

# Hello hardness, my old friend

**Theorem 5.4 (para-NP-complete → NP-hard for bounded parameter)**
Let $(\Pi, \kappa)$ be a nontrivial ($\emptyset \neq Y \neq L$) parameterized problem that is para-NP-complete. Then there is a $d \in \mathbb{N}$ such that $\Pi_{\kappa \leq d}$ ($\Pi$ restricted to instances with $\kappa(x) \leq d$) is NP-hard. ◀

The converse is essentially also true (using a generalization of kernelizations).

*Proof:* eventually want $\Pi' \leq_p \Pi_{\kappa \leq d}$

Let $\Pi' = (L', Y', \Sigma')$ be any NP-complete problem and $\kappa_{\text{one}} \equiv 1$.

A nondeterministic polytime algorithm for $\Pi'$ is a nondeterministic fpt-algorithm for $(\Pi', \kappa_{\text{one}}) \rightsquigarrow (\Pi', \kappa_{\text{one}}) \in$ para-NP.

# Hello hardness, my old friend

**Theorem 5.4 (para-NP-complete → NP-hard for bounded parameter)**

Let $(\Pi, \kappa)$ be a nontrivial ($\emptyset \neq Y \neq L$) parameterized problem that is para-NP-complete. Then there is a $d \in \mathbb{N}$ such that $\Pi_{\kappa \leq d}$ ($\Pi$ restricted to instances with $\kappa(x) \leq d$) is NP-hard. ◀

The converse is essentially also true (using a generalization of kernelizations).

*Proof:*

Let $\Pi' = (L', Y', \Sigma')$ be any NP-complete problem and $\kappa_{\text{one}} \equiv 1$.

A nondeterministic polytime algorithm for $\Pi'$ is a nondeterministic fpt-algorithm for $(\Pi', \kappa_{\text{one}}) \rightsquigarrow (\Pi', \kappa_{\text{one}}) \in$ para-NP.

$(\Pi, \kappa)$ para-NP-hard $\rightsquigarrow$ $(\Pi', \kappa_{\text{one}}) \leq_{fpt} (\Pi, \kappa)$ via some $A$ with

- $x \in Y' \iff A(x) \in Y$,
- running time $f(\kappa_{\text{one}}(x)) \cdot |x|^c = f(1) \cdot |x|^c \rightsquigarrow$ polytime,
- $\kappa(A(x)) \leq g(\kappa_{\text{one}}(x)) = g(1) =: d$.

# Hello hardness, my old friend

**Theorem 5.4 (para-NP-complete → NP-hard for bounded parameter)**
Let $(\Pi, \kappa)$ be a nontrivial ($\emptyset \neq Y \neq L$) parameterized problem that is para-NP-complete. Then there is a $d \in \mathbb{N}$ such that $\Pi_{\kappa \leq d}$ ($\Pi$ restricted to instances with $\kappa(x) \leq d$) is NP-hard. ◀

The converse is essentially also true (using a generalization of kernelizations).

*Proof:*

Let $\Pi' = (L', Y', \Sigma')$ be any NP-complete problem and $\kappa_{\text{one}} \equiv 1$.

A nondeterministic polytime algorithm for $\Pi'$ is a nondeterministic fpt-algorithm for $(\Pi', \kappa_{\text{one}}) \rightsquigarrow (\Pi', \kappa_{\text{one}}) \in$ para-NP.

$(\Pi, \kappa)$ para-NP-hard $\rightsquigarrow$ $(\Pi', \kappa_{\text{one}}) \leq_{fpt} (\Pi, \kappa)$ via some $A$ with

- $x \in Y' \iff A(x) \in Y$,
- running time $f(\kappa_{\text{one}}(x)) \cdot |x|^c = f(1) \cdot |x|^c \rightsquigarrow$ polytime,
- $\kappa(A(x)) \leq g(\kappa_{\text{one}}(x)) = g(1) =: d$.

$\rightsquigarrow$ $A$ is a polytime reduction $\Pi' \leq_p \Pi_{\kappa \leq d} \rightsquigarrow$ $\Pi_{\kappa \leq d}$ is NP-hard. ■

# para-NP-complete is too strict

Above Theorem means that many problems cannot be para-NP-complete!

For each of the following

- $p$-Clique,
- $p$-IndependentSet
- $p$-DominatingSet

bounding $k$ by a **constant** $d$ makes *polytime* algorithm possible.

# para-NP-complete is too strict

Above Theorem means that many problems cannot be para-NP-complete!

For each of the following

- $p$-Clique,
- $p$-IndependentSet
- $p$-DominatingSet

bounding $k$ by a **constant** $d$ makes *polytime* algorithm possible.

$\rightsquigarrow$ $\Pi_{\kappa \leq d}$ cannot be NP-hard for each of these (unless P = NP)

- but we rather expect them $\notin$ FPT

$\rightsquigarrow$ para-NP theory does not settle complexity status

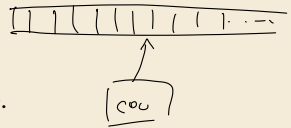
## 5.3 Bounded Nondeterminism: W[P]

# Bye bye, TM

para-NP is too large a class to have interesting complete problems

$\rightsquigarrow$ We must weaken the class. Unfortunately, TM inconvenient here.

# Bye bye, TM




para-NP is too large a class to have interesting complete problems
⇝ We must weaken the class. Unfortunately, TM inconvenient here.

≠ word-RAM  w-bit numbers

**Definition 5.5 (Nondeterministic RAM (NRAM), $\kappa$-restricted)**
An NRAM $M$ is a (unit-cost) RAM with the additional operation to nondeterministically *guess* a number between 0 and a current register content.

An NRAM $M$ that decides a parameterized problem $(\Pi, \kappa)$, $\Pi = (L, Y)$ is *$\kappa$-restricted* if on every input $x \in L$ with $n = |x|$ and $k = \kappa(x)$, *every* computation

1. performs at most $f(k) \cdot p(n)$ steps,
2. *at most $g(k)$ of them nondeterministic,*
3. uses at most $f(k) \cdot p(n)$ memory cells, and
4. those never contain numbers larger than $f(k) \cdot p(n)$,

= MaxInt

for computable functions $f$ and $g$ and a polynomial $p$.

◀

# W[P]

**Definition 5.6 (W[$P$])**

The class W[$P$] is the set of all parameterized problems $(\Pi, \kappa)$ decidable by a $\kappa$-restricted NRAM. ◀

# W[P]

## Definition 5.6 (W[$P$])

The class W[$P$] is the set of all parameterized problems $(\Pi, \kappa)$ decidable by a $\kappa$-restricted NRAM. ◀

**Example:** $p$-Clique $\in$ W[$P$]

- brute force does it
  - nondeterministically guess $k$-set $S$ of vertices  $k$ non-det. steps, numbers in $[n]$
  - check if $G[S]$ is a clique
  - $\rightsquigarrow$ $O(n^2 + k + k^2)$ total time

# A first W[$P$]-complete problem?

Define hardness and completeness for W[$P$] using $\leq_{fpt}$.

What could be the mother of all W[$P$]-complete problems?

Some parameterized version of SAT? Parameter #variables does not work. (Why?)

Can p-variables-SAT be W[P]-complete?

$\Uparrow$ FPT

non-det guess assignment of variables

$2^k \cdot n^{O(1)}$

# A first W[$P$]-complete problem?

Define hardness and completeness for W[$P$] using $\leq_{fpt}$.

What could be the mother of all W[$P$]-complete problems?

Some parameterized version of SAT? Parameter #variables does not work. (Why?)

- What can be guessed using $k$ numbers in $[n]$?

# A first W[$P$]-complete problem?

Define hardness and completeness for W[$P$] using $\leq_{fpt}$.

What could be the mother of all W[$P$]-complete problems?

Some parameterized version of SAT? Parameter #variables does not work. (Why?)

- What can be guessed using $k$ numbers in $[n]$?

$\rightsquigarrow$ A subset of variables of *size* $k$!

# Weighted SAT

**Definition 5.7 (Weighted Satisfiability)**

Given: Boolean formula $\varphi$ and integer $k \in \mathbb{N}$

Parameter: $k$

Question: $\exists$ satisfying assignment with weight $= k$ ?

Recall: weight = #*true* variables

◀

# Weighted SAT

for Circuits

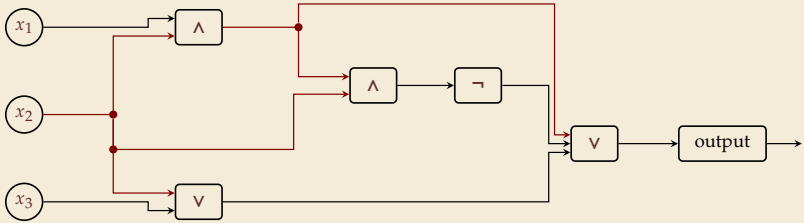
## Definition 5.7 (Weighted Satisfiability)

Given: Boolean formula $\varphi$ and integer $k \in \mathbb{N}$

Parameter: $k$ circuit

Question: $\exists$ satisfying assignment with weight $= k$ ?

Recall: weight = #*true* variables




a *circuit*: gates may be reused (DAG);
a *formula* is a circuit with all fan-outs 1 (tree)

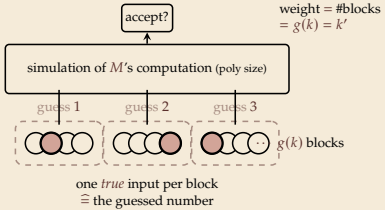
## Theorem 5.8 ($p$-WSAT(CIRC) is W[$P$]-complete)

The weighted satisfiability problem for boolean **circuits** parameterized by the weight is W[$P$]-complete.

# Weighted SAT – Proof

*Proof (sketch):*

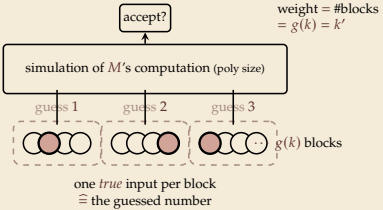
- **"$\in$ W[$P$]"**: $\kappa$-restricted NRAM can guess $k$ indices of *true* inputs ($k$ numbers $\leq n$) and evaluate the circuit bottom-up in polytime ✓



# Weighted SAT – Proof

*Proof (sketch):*

- **"$\in$ W[$P$]"**: $\kappa$-restricted NRAM can guess $k$ indices of *true* inputs ($k$ numbers $\leq n$) and evaluate the circuit bottom-up in polytime ✓
- **"W[$P$]-hard"**: $(\Pi, \kappa) \in \mathsf{W}[P] \implies (\Pi, \kappa) \leq_{fpt} p\text{-WSAT(CIRC)}$
  - simulate $\kappa$-restricted NRAM $M$ on **hardwired** input $x$ as circuit $C_x$:
  - all computations $\leq f(k)p(n)$ steps, memory cells $\leq \lg\big(f(k)p(n)\big)$ bits
    - $\rightsquigarrow$ deterministic part simulated by circuit of size $\big(f(k)p(n)\big)^{O(1)} = f'(k) \cdot \mathrm{poly}(n)$
  - only $g(k)$ guesses are unknowns: encode in *unary* by $f(k)p(n)$ inputs, **exactly 1** *true*
  - $\rightsquigarrow$ $C_x$ outputs 1 iff $M$ accepts with given guesses



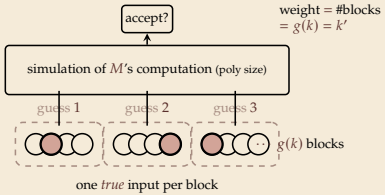
# Weighted SAT – Proof

*Proof (sketch):*

- **"$\in$ W[$P$]"**: $\kappa$-restricted NRAM can guess $k$ indices of *true* inputs ($k$ numbers $\leq n$) and evaluate the circuit bottom-up in polytime ✓
- **"W[$P$]-hard"**: $(\Pi, \kappa) \in \mathsf{W}[P] \implies (\Pi, \kappa) \leq_{\mathit{fpt}} p\text{-WSAT(CIRC)}$
  - simulate $\kappa$-restricted NRAM $M$ on **hardwired** input $x$ as circuit $C_x$:
  - all computations $\leq f(k)p(n)$ steps, memory cells $\leq \lg\big(f(k)p(n)\big)$ bits
    - $\rightsquigarrow$ deterministic part simulated by circuit of size $\big(f(k)p(n)\big)^{O(1)} = f'(k) \cdot \mathrm{poly}(n)$
  - only $g(k)$ guesses are unknowns: encode in *unary* by $f(k)p(n)$ inputs, **exactly 1** *true*
  - $\rightsquigarrow$ $C_x$ outputs 1 iff $M$ accepts with given guesses

$\rightsquigarrow$ $(x, k) \mapsto (C_x, g(k))$ is an fpt-reduction to $p$-WSAT(CIRC). ■

***Note:*** *Simulation needs circuits; simulation via formula needs intermediate variables, which increase the weight*



## 5.4 W[1]-Hardness

# Tail-nondeterminism

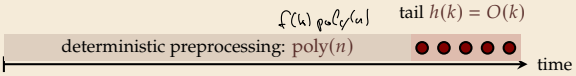
Circuit satisfiability still too strong to show hardness of many interesting problems.
$\rightsquigarrow$ We must weaken the class *further*.

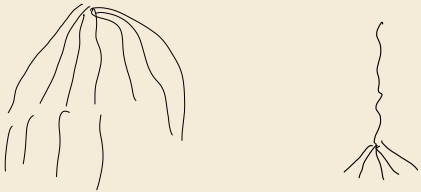
# Tail-nondeterminism

Circuit satisfiability still too strong to show hardness of many interesting problems.
$\rightsquigarrow$ We must weaken the class *further*.

## Definition 5.9 (tail-nondeterministic NRAM)

A $\kappa$-restricted NRAM $M$ for a problem $(\Pi, \kappa)$ is called *tail-nondeterministic* if all nondeterministic steps occur only among the last $h(\kappa(x))$ steps. ◀



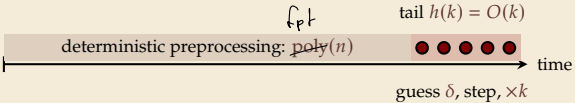



# Tail-nondeterminism

Circuit satisfiability still too strong to show hardness of many interesting problems.
$\rightsquigarrow$ We must weaken the class *further*.

## Definition 5.9 (tail-nondeterministic NRAM)

A $\kappa$-restricted NRAM $M$ for a problem $(\Pi, \kappa)$ is called *tail-nondeterministic* if all nondeterministic steps occur only among the last $h(\kappa(x))$ steps. ◀




## Definition 5.10 (W[1])

The class W[1] consists of all parameterized decision problems $(\Pi, \kappa)$ that are decided by a tail-nondeterministic $\kappa$-restricted NRAM. ◀

As before, define hardness and completeness for W[1] w.r.t. $\leq_{fpt}$.

# The Parameterized Landscape

$$\mathsf{FPT} \subseteq \mathsf{W}[1] \subseteq \mathsf{W}[2] \subseteq \cdots \subseteq \mathsf{W}[P] \subseteq \mathsf{XP} \cap \mathsf{para\text{-}NP}$$

- XP: problems solvable in time $n^{f(k)}$ ("polytime for every fixed $k$");
  FPT $\subsetneq$ XP is *provable* (time hierarchy)
- FPT $\subseteq$ W[1]: an fpt-algorithm is a $\kappa$-restricted NRAM with $0$ guesses
- W[1] $\subseteq$ W[$P$]: tail-nondeterministic is a special case
- W[$P$] $\subseteq$ para-NP: a $\kappa$-restricted NRAM is a nondeterministic fpt-algorithm
- W[$P$] $\subseteq$ XP: try all $\leq (f(k)p(n))^{g(k)} = n^{O(g(k))}$ guess sequences

$\rightsquigarrow$ **all inclusions are believed strict**, none is proven except FPT $\subsetneq$ XP.

Hardness for W[1] (or any class above it) is the standard evidence against fpt-algorithms;
the classical support "FPT = W[1] $\Rightarrow$ ETH fails" is void since ETH was refuted (Oct 2026, next unit).

# Stop

## Definition 5.11 ($p$-ShortTuringMachineAcceptance)

Given: A nondeterministic (single-tape) Turing machine $M$, an input $x$ and $k \in \mathbb{N}$ be given.
Parameter: $k$
Question: Does $M$ accepts $x$ after at most $k$ computation steps?

normally: TM nondet. used for guesses

like $x_i$ true or false

guesses only of $|T| \cdot |Q| \cdot 3$

for fixed TM some constant

$C^k$ would be fpt

# Stop

**Definition 5.11 ($p$-ShortTuringMachineAcceptance)**

Given: A nondeterministic (single-tape) Turing machine $M$, an input $x$ and $k \in \mathbb{N}$ be given.
Parameter: $k$
Question: Does $M$ accepts $x$ after at most $k$ computation steps? ◀

- $M$ is part of input, so state space and tape alphabet are not fixed!

⇝ up to $n$ different non-deterministic choices in *each* step. ($n$ is size of encoding of $M$)

⇝ Trivial algorithm has to simulate up to $n^{k+1}$ steps of $M$.

- Equivalent here to halting problem for $x = \varepsilon$, since we can hard-wire the given input into the states of a TM $M'$ constructed from $M$.

# p-ShortTuringMachineAcceptance is W[1]-complete

**Theorem 5.12**

*p*-ShortTuringMachineAcceptance is W[1]-complete.

*Proof (Sketch):*

**∈ W[1]:** Given TM $M$, input $x$, number of steps $k$.
We construct tail-nondeterministic NRAM $A$ to decide if $M$ halts.
$A$ simulated $M$ for $k$ steps (and give its output).
$\rightsquigarrow$ $k$ nondeterministic steps, all within the last $O(k)$ steps.

# p-ShortTuringMachineAcceptance is W[1]-complete

**Theorem 5.12**

$p$-ShortTuringMachineAcceptance is W[1]-complete. ◀

*Proof (Sketch):*

**∈ W[1]:** Given TM $M$, input $x$, number of steps $k$.
We construct tail-nondeterministic NRAM $A$ to decide if $M$ halts.
$A$ simulated $M$ for $k$ steps (and give its output).
$\rightsquigarrow$ $k$ nondeterministic steps, all within the last $O(k)$ steps.

**W[1]-hard:** $(\Pi, \kappa) \in \mathsf{W}[1]$ $\rightsquigarrow$ $\exists$ tail-nondet. $\kappa$-restricted NRAM $A$, tail length $h(k)$.
On input $y$: simulate deterministic prefix of $A(y)$, reaching config $c$.
Tail touches $\leq h(k)$ memory cells, uses $\leq g(k)$ guesses of numbers $\leq f(k)p(n)$.

# p-ShortTuringMachineAcceptance is W[1]-complete

**Theorem 5.12**

*p*-ShortTuringMachineAcceptance is W[1]-complete. ◀

*Proof (Sketch):*

**∈ W[1]:** Given TM $M$, input $x$, number of steps $k$.
We construct tail-nondeterministic NRAM $A$ to decide if $M$ halts.
$A$ simulated $M$ for $k$ steps (and give its output).
$\rightsquigarrow$ $k$ nondeterministic steps, all within the last $O(k)$ steps.

**W[1]-hard:** $(\Pi, \kappa) \in \mathsf{W}[1]$ $\rightsquigarrow$ $\exists$ tail-nondet. $\kappa$-restricted NRAM $A$, tail length $h(k)$.
On input $y$: simulate deterministic prefix of $A(y)$, reaching config $c$.
Tail touches $\leq h(k)$ memory cells, uses $\leq g(k)$ guesses of numbers $\leq f(k)p(n)$.

Build a nondet. TM $M_y$ whose states and tape alphabet encode memory contents; nondeterminism simulates guesses; ($y$ hardcoded into TM)

$\rightsquigarrow$ $M_y$ accepts in $k' = O(h(k))$ steps iff some tail computation of $A$ accepts.
$\rightsquigarrow$ $y \mapsto (M_y, \varepsilon, k')$ is an fpt-reduction. ■